Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Информационные системы

Курсовая работа. Этап 2

Информационная система поиска салонов красоты
и онлайн-записи на услуги

Проектирование и реализация базы данных PostgreSQL

Выполнили:
Чэнь Хаолинь P3216 407960
Павлов Эдгар P3316 409297
Преподаватель:
Коновалов Арсений Антонович

г. Санкт-Петербург, 2026

Содержание

1 Задание и границы системы **3**

2 ER-модель **4**

3 Даталогическая модель **5**

4 Целостность и доступ **6**

5 Критические функции и процедуры **11**

6 Индексы по прецедентам **12**

7 Заключение **13**

1. Задание и границы системы

Цель этапа --- спроектировать и реализовать базу данных платформы салонов красоты. Требуется:

1. Построить ER-модель не менее чем из десяти сущностей со связью M:N.
2. Согласовать модель с преподавателем и получить даталогическую модель.
3. Реализовать её в PostgreSQL, обеспечить целостность DDL и триггерами.
4. Подготовить скрипты жизненного цикла базы, демоданные и бизнес-операции PL/pgSQL.
5. Обосновать индексы по прецедентам и представить результаты проверки.

Платформа обслуживает несколько салонов одного города. У пользователя одна роль: клиент, мастер, владелец или администратор. Владелец управляет одним салоном, мастер работает в одном салоне. Запись содержит одну услугу и одного мастера. Денежные значения заданы в рублях, локальное время --- Europe/Moscow. Горизонт записи составляет 30 календарных дней, включая текущий; начала выбираются с шагом 15 минут, длительность услуги может быть произвольным положительным числом минут.

2. ER-модель

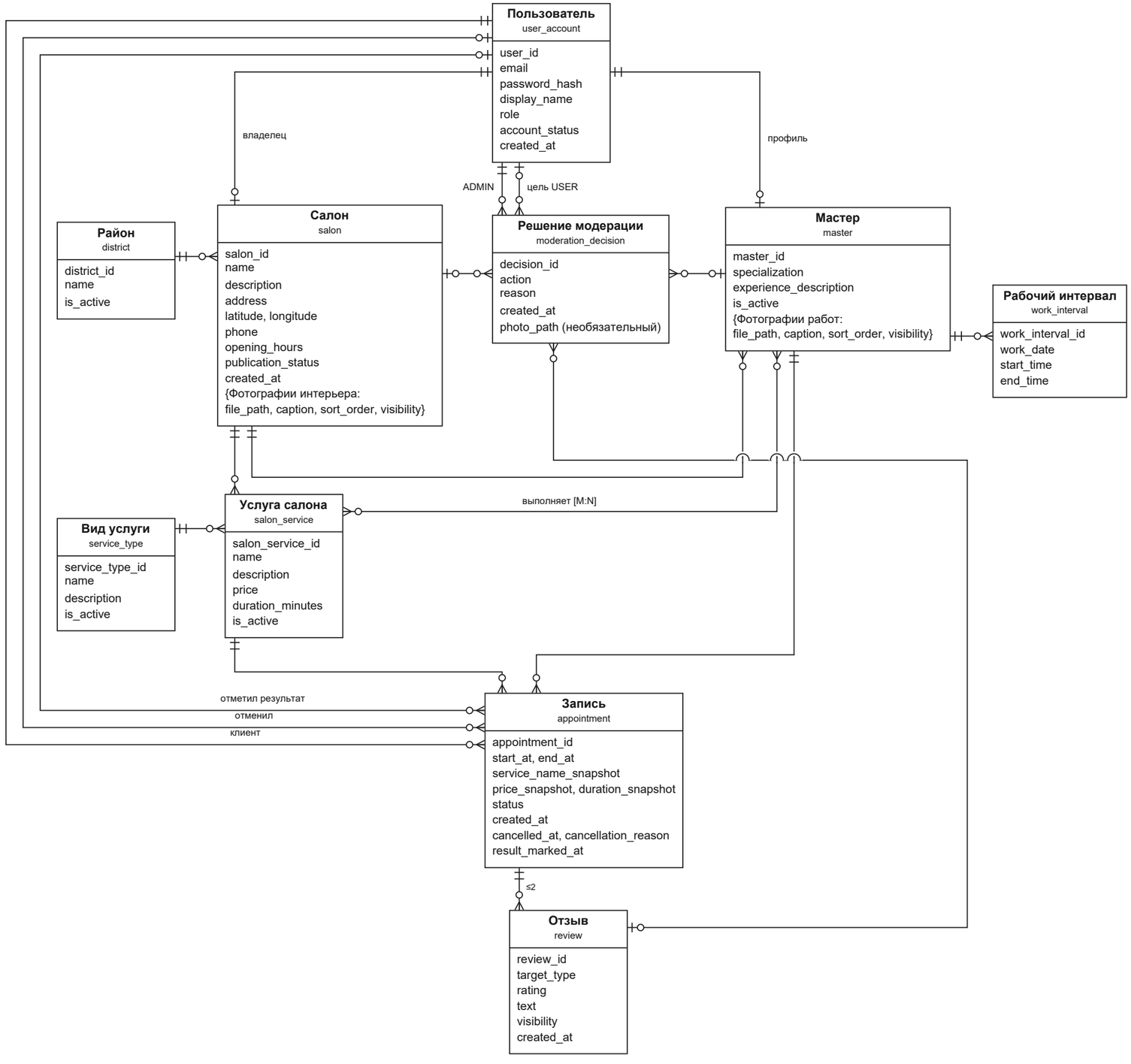


Рисунок 1. Полная ER-модель: 10 сущностей и прямая связь M:N.

3. Даталогическая модель

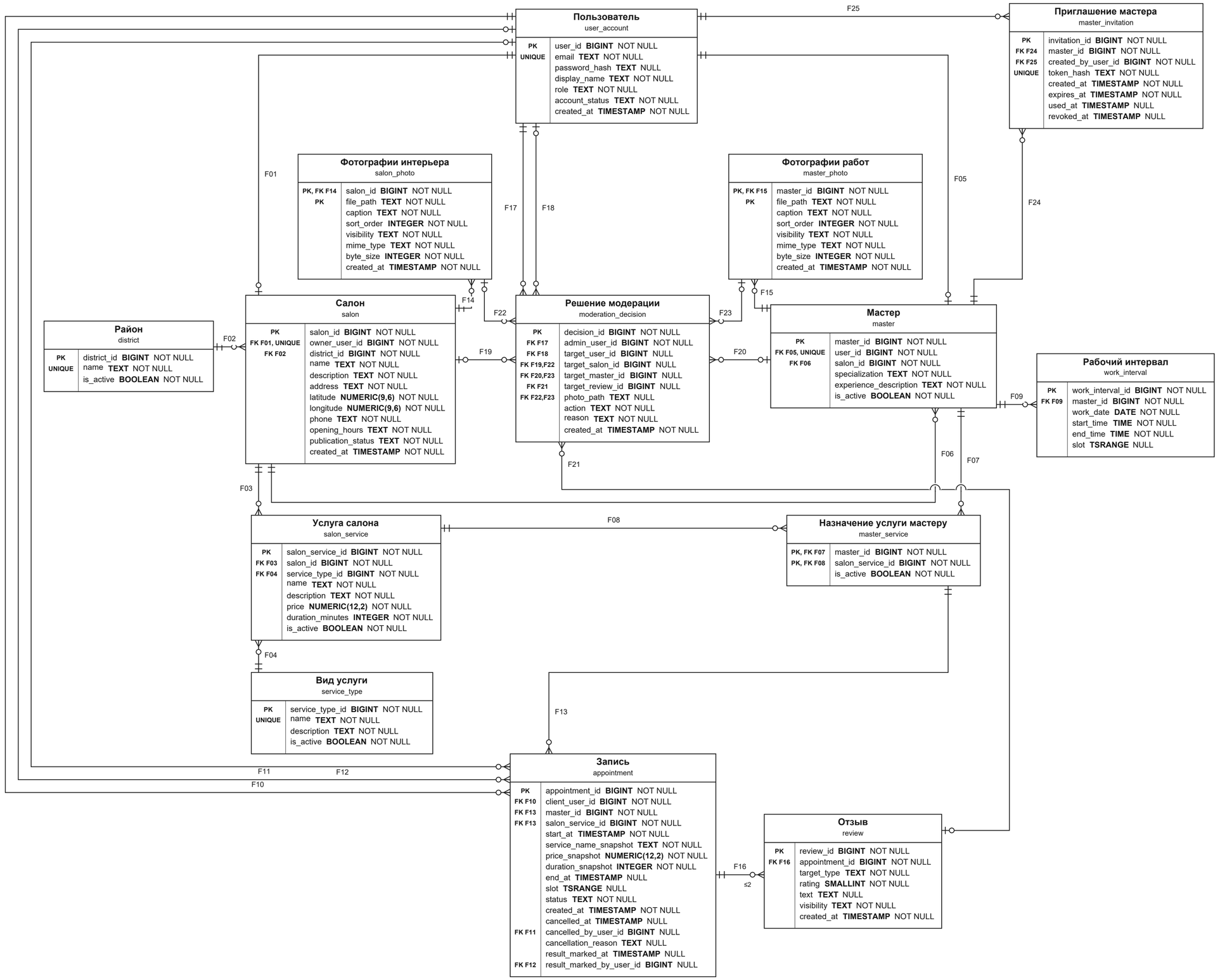


Рисунок 2. Полная даталогическая модель PostgreSQL: 14 таблиц и 25 внешних ключей.

4. Целостность и доступ

4.1. Создание таблиц

```
CREATE EXTENSION IF NOT EXISTS btree_gist WITH SCHEMA public;
CREATE EXTENSION IF NOT EXISTS pgcrypto WITH SCHEMA public;
CREATE SCHEMA beauty;
REVOKE ALL ON SCHEMA beauty FROM PUBLIC;
CREATE FUNCTION beauty.local_now() RETURNS timestamp LANGUAGE sql VOLATILE
SET search_path = pg_catalog AS $$ SELECT clock_timestamp() AT TIME ZONE
    'Europe/Moscow' $$;
```

Пользователь (`user_account`)

```
CREATE TABLE beauty.user_account (
 user_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
 email text NOT NULL CHECK (email = lower(btrim(email)) AND email LIKE '%@%'),
 password_hash text,
 display_name text NOT NULL CHECK (btrim(display_name) <> ''),
 role text NOT NULL CHECK (role IN ('CLIENT', 'MASTER', 'OWNER', 'ADMIN')),
 account_status text NOT NULL DEFAULT 'ACTIVE' CHECK (account_status IN ('INVITED',
    'ACTIVE', 'BLOCKED', 'ARCHIVED')),
 created_at timestamp NOT NULL DEFAULT beauty.local_now(),
 CONSTRAINT uq_user_email UNIQUE(email),
 CONSTRAINT ck_invited_role CHECK(account_status<>'INVITED' OR role='MASTER'),
 CONSTRAINT ck_password CHECK (password_hash IS NOT NULL OR (role='MASTER' AND
    account_status='INVITED'))
);
```

Район (`district`)

```
CREATE TABLE beauty.district (
 district_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
 name text NOT NULL UNIQUE CHECK(btrim(name)<>''), is_active boolean NOT NULL DEFAULT
    true
);
```

Салон (`salon`)

```
CREATE TABLE beauty.salon (
 salon_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
 owner_user_id bigint NOT NULL UNIQUE REFERENCES beauty.user_account ON DELETE
    RESTRICT,
 district_id bigint NOT NULL REFERENCES beauty.district ON DELETE RESTRICT,
 name text NOT NULL CHECK(btrim(name)<>''), description text NOT NULL DEFAULT '',
 address text NOT NULL CHECK(btrim(address)<>''),
 latitude numeric(9, 6) NOT NULL CHECK(latitude BETWEEN -90 AND 90),
 longitude numeric(9, 6) NOT NULL CHECK(longitude BETWEEN -180 AND 180),
 phone text NOT NULL, opening_hours text NOT NULL,
 publication_status text NOT NULL DEFAULT 'PENDING'
   CHECK(publication_status IN ('PENDING', 'RETURNED', 'PUBLISHED', 'UNPUBLISHED',
    'BLOCKED', 'ARCHIVED')),
 created_at timestamp NOT NULL DEFAULT beauty.local_now()
);
```

Вид услуги (`service_type`)

```
CREATE TABLE beauty.service_type (
 service_type_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
 name text NOT NULL UNIQUE CHECK(btrim(name)<>''), description text NOT NULL DEFAULT
    '',
 is_active boolean NOT NULL DEFAULT true
);
```

Услуга салона (`salon_service`)

```
CREATE TABLE beauty.salon_service (
 salon_service_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
 salon_id bigint NOT NULL REFERENCES beauty.salon ON DELETE RESTRICT,
 service_type_id bigint NOT NULL REFERENCES beauty.service_type ON DELETE RESTRICT,
 name text NOT NULL CHECK(btrim(name)<>''), description text NOT NULL DEFAULT '',
 price numeric(12, 2) NOT NULL CHECK(price>=0),
 duration_minutes integer NOT NULL CHECK(duration_minutes>0),
 is_active boolean NOT NULL DEFAULT true
);
```

Мастер (`master`)

```
CREATE TABLE beauty.master (
 master_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
 user_id bigint NOT NULL UNIQUE REFERENCES beauty.user_account ON DELETE RESTRICT,
 salon_id bigint NOT NULL REFERENCES beauty.salon ON DELETE RESTRICT,
 specialization text NOT NULL DEFAULT '', experience_description text NOT NULL DEFAULT
    '',
 is_active boolean NOT NULL DEFAULT true
);
```

Назначение услуги мастеру (`master_service`)

```
CREATE TABLE beauty.master_service (
 master_id bigint NOT NULL REFERENCES beauty.master ON DELETE RESTRICT,
 salon_service_id bigint NOT NULL REFERENCES beauty.salon_service ON DELETE RESTRICT,
 is_active boolean NOT NULL DEFAULT true,
 PRIMARY KEY(master_id, salon_service_id)
);
```

Рабочий интервал (`work_interval`)

```
CREATE TABLE beauty.work_interval (
 work_interval_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
 master_id bigint NOT NULL REFERENCES beauty.master ON DELETE RESTRICT,
 work_date date NOT NULL, start_time time NOT NULL, end_time time NOT NULL,
 slot tsrange GENERATED ALWAYS AS (tsrange(work_date+start_time, work_date+end_time,
    '[)')) STORED,
 CHECK(start_time<end_time),
 CONSTRAINT ex_work_overlap EXCLUDE USING gist(master_id WITH =, slot WITH &&)
);
```

Запись (appointment)

```
CREATE TABLE beauty.appointment (
 appointment_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
 client_user_id bigint NOT NULL REFERENCES beauty.user_account ON DELETE RESTRICT,
 master_id bigint NOT NULL, salon_service_id bigint NOT NULL,
 start_at timestamp NOT NULL,
 service_name_snapshot text NOT NULL,
 price_snapshot numeric(12, 2) NOT NULL CHECK(price_snapshot>=0),
 duration_snapshot integer NOT NULL CHECK(duration_snapshot>0),
 end_at timestamp GENERATED ALWAYS AS
    (start_at+make_interval(mins=>duration_snapshot)) STORED,
 slot tsrange GENERATED ALWAYS AS (tsrange(start_at,
    start_at+make_interval(mins=>duration_snapshot), '[)')) STORED,
 status text NOT NULL DEFAULT 'CONFIRMED' CHECK(status IN ('CONFIRMED', 'CANCELLED',
    'COMPLETED', 'NO_SHOW')),
 created_at timestamp NOT NULL DEFAULT beauty.local_now(),
 cancelled_at timestamp, cancelled_by_user_id bigint REFERENCES beauty.user_account ON
    DELETE RESTRICT,
 cancellation_reason text,
 result_marked_at timestamp, result_marked_by_user_id bigint REFERENCES
    beauty.user_account ON DELETE RESTRICT,
 FOREIGN KEY(master_id, salon_service_id) REFERENCES beauty.master_service ON DELETE
    RESTRICT,
 CONSTRAINT ck_start_grid CHECK(extract(minute FROM start_at)::integer % 15=0 AND
    extract(second FROM start_at)=0),
 CONSTRAINT ck_appointment_evidence CHECK (
  (status='CONFIRMED' AND cancelled_at IS NULL AND cancelled_by_user_id IS NULL AND
    cancellation_reason IS NULL
    AND result_marked_at IS NULL AND result_marked_by_user_id IS NULL) OR
  (status='CANCELLED' AND cancelled_at IS NOT NULL AND cancelled_by_user_id IS NOT NULL
    AND result_marked_at IS NULL AND result_marked_by_user_id IS NULL) OR
  (status IN ('COMPLETED', 'NO_SHOW') AND result_marked_at IS NOT NULL AND
    result_marked_by_user_id IS NOT NULL
    AND cancelled_at IS NULL AND cancelled_by_user_id IS NULL AND cancellation_reason
    IS NULL)
 ),
 CONSTRAINT ex_appointment_overlap EXCLUDE USING gist(master_id WITH =, slot WITH &&)
    WHERE(status<>'CANCELLED')
);
```

Фотографии интерьера (salon_photo)

```
CREATE TABLE beauty.salon_photo (
 salon_id bigint NOT NULL REFERENCES beauty.salon ON DELETE RESTRICT,
 file_path text NOT NULL CHECK(btrim(file_path)<>''), caption text NOT NULL DEFAULT '',
 sort_order integer NOT NULL DEFAULT 0 CHECK(sort_order>=0),
 visibility text NOT NULL DEFAULT 'PUBLISHED' CHECK(visibility IN ('PUBLISHED',
    'HIDDEN', 'ARCHIVED')),
 mime_type text NOT NULL CHECK(mime_type IN ('image/jpeg', 'image/png')),
 byte_size integer NOT NULL CHECK(byte_size BETWEEN 1 AND 5242880),
 created_at timestamp NOT NULL DEFAULT beauty.local_now(), PRIMARY KEY(salon_id,
    file_path)
);
```

Фотографии работ (`master_photo`)

```
CREATE TABLE beauty.master_photo (
 master_id bigint NOT NULL REFERENCES beauty.master ON DELETE RESTRICT,
 file_path text NOT NULL CHECK(btrim(file_path)<>''), caption text NOT NULL DEFAULT '',
 sort_order integer NOT NULL DEFAULT 0 CHECK(sort_order>=0),
 visibility text NOT NULL DEFAULT 'PUBLISHED' CHECK(visibility IN ('PUBLISHED',
    'HIDDEN', 'ARCHIVED')),
 mime_type text NOT NULL CHECK(mime_type IN ('image/jpeg', 'image/png')),
 byte_size integer NOT NULL CHECK(byte_size BETWEEN 1 AND 5242880),
 created_at timestamp NOT NULL DEFAULT beauty.local_now(), PRIMARY KEY(master_id,
    file_path)
);
```

Отзыв (`review`)

```
CREATE TABLE beauty.review (
 review_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
 appointment_id bigint NOT NULL REFERENCES beauty.appointment ON DELETE RESTRICT,
 target_type text NOT NULL CHECK(target_type IN ('SALON', 'MASTER')),
 rating smallint NOT NULL CHECK(rating BETWEEN 1 AND 5), text text,
 visibility text NOT NULL DEFAULT 'PUBLISHED' CHECK(visibility IN ('PUBLISHED',
    'HIDDEN')),
 created_at timestamp NOT NULL DEFAULT beauty.local_now(),
 UNIQUE(appointment_id, target_type)
);
```

Решение модерации (`moderation_decision`)

```
CREATE TABLE beauty.moderation_decision (
 decision_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
 admin_user_id bigint NOT NULL REFERENCES beauty.user_account ON DELETE RESTRICT,
 target_user_id bigint REFERENCES beauty.user_account ON DELETE RESTRICT,
 target_salon_id bigint REFERENCES beauty.salon ON DELETE RESTRICT,
 target_master_id bigint REFERENCES beauty.master ON DELETE RESTRICT,
 target_review_id bigint REFERENCES beauty.review ON DELETE RESTRICT,
 photo_path text,
 action text NOT NULL CHECK(action IN ('APPROVE', 'RETURN', 'PUBLISH', 'UNPUBLISH',
    'BLOCK', 'UNBLOCK', 'HIDE', 'RESTORE')),
 reason text NOT NULL CHECK(btrim(reason)<>''), created_at timestamp NOT NULL DEFAULT
    beauty.local_now(),
 CHECK(num_nonnulls(target_user_id, target_salon_id, target_master_id,
    target_review_id)=1),
 CHECK(photo_path IS NULL OR (btrim(photo_path)<>'' AND num_nonnulls(target_salon_id,
    target_master_id)=1)),
 FOREIGN KEY(target_salon_id, photo_path) REFERENCES beauty.salon_photo(salon_id,
    file_path) ON DELETE RESTRICT,
 FOREIGN KEY(target_master_id, photo_path) REFERENCES beauty.master_photo(master_id,
    file_path) ON DELETE RESTRICT
);
```

Приглашение мастера (`master_invitation`)

```
CREATE TABLE beauty.master_invitation (
 invitation_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
 master_id bigint NOT NULL REFERENCES beauty.master ON DELETE RESTRICT,
 created_by_user_id bigint NOT NULL REFERENCES beauty.user_account ON DELETE RESTRICT,
 token_hash text NOT NULL UNIQUE,
 created_at timestamp NOT NULL DEFAULT beauty.local_now(), expires_at timestamp NOT
    NULL,
 used_at timestamp, revoked_at timestamp,
 CHECK(expires_at>created_at), CHECK(num_nonnulls(used_at, revoked_at)<=1),
 CHECK(used_at IS NULL OR (used_at>=created_at AND used_at<expires_at)),
 CHECK(revoked_at IS NULL OR revoked_at>=created_at)
);
```

4.2. Контроль бизнес-правил

Инвариант	Реализация
Идентичность и принадлежность	PK, FK, UNIQUE; неизменность роли и владельческих связей; проверка ролей и одного салона триггерами.
Нет пересечения времени	GiST EXCLUDE по мастеру и диапазону для рабочих интервалов и неотменённых записей. Полуоткрытые интервалы допускают соседние посещения.
Запись допустима	Триггер проверяет публикацию, активность, разрешённую пару, горизонт, шаг начала и вхождение всей услуги в один рабочий интервал.
История и статусы	Снимки неизменны; отмена до начала, результат после окончания. Сокращение графика и деактивация не нарушают ещё не завершённые подтверждённые записи.
Отзывы и модерация	Только COMPLETED, рейтинг 1--5, UNIQUE по типу цели; неизменяемый текст и аудит. CHECK XOR и составные FK для фото.
Фото и приглашения	До 20 неархивных фото; JPEG/PNG до 5 МБ; скрытые фото учитываются в квоте. Уникальный одноразовый токен, срок и взаимное исключение использования и отзыва.

Триггеры защищают условия, зависящие от других строк и времени. Например, контроль права на отзыв включает следующую проверку:

```
IF NOT EXISTS (
  SELECT FROM beauty.appointment
  WHERE appointment_id = NEW.appointment_id
    AND status = 'COMPLETED'
) THEN
  RAISE EXCEPTION 'Review requires a completed visit'
    USING ERRCODE = '23514';
END IF;
```

Все бизнес-записи получают единый транзакционный advisory lock до чтения изменяемых условий и блокировки строк. Допускается READ COMMITTED; иной уровень явно отклоняется. Это устраняет проверяемые гонки между записью, графиком, каталогом и токенами, но последовательно выполняет изменения всей платформы. Производительность при 20 параллельных клиентах не подтверждена; для масштабирования требуется более мелкая схема блокировок.

Backend получает только EXECUTE на API; актор берётся из аутентифицированной сессии. Проверку файлов выполняет файловый сервис.

5. Критические функции и процедуры

Реализованы 22 бизнес-операции. Пишущие операции используют PL/pgSQL; выбор свободного времени выражен SQL-запросом. Два view рассчитывают независимые рейтинги салона и мастера только по опубликованным отзывам.

Прецеденты	API и назначение
П1	`register_user`, `authenticate`: регистрация и пароль; `accept_invitation`: активация мастера.
П2--П4, П12	`search_salons`, `available_slots`, `book_appointment`: фильтры, допустимые начала и подтверждение условий.
П5, П9, П10	`appointment_journal`, `cancel_appointment`, `mark_visit`: личная история, отмена и результат посещения.
П6, П7, П11	`add_review`, `moderate`, `create_salon`, `resubmit_salon`: отзывы, решения и публикация.
П8, П9	`invite_master`, `reissue_invitation`, `set_master_service`, `save_work_interval`, `delete_work_interval`, `save_service`, `save_master_profile`, `put_photo`, `archive_photo`: управление своим салоном и профилем.

При подтверждении записи сервер повторно проверяет цену и длительность: изменение каталога требует нового подтверждения клиента. После INSERT триггер проверяет доступность и сохраняет актуальные снимки. Полный пример критической функции:

```
CREATE FUNCTION beauty.book_appointment(
  p_client bigint, p_master bigint, p_service bigint,
  p_start timestamp, p_expected_price numeric,
  p_expected_duration integer
) RETURNS bigint
LANGUAGE plpgsql SECURITY DEFINER
SET search_path = pg_catalog, beauty, pg_temp AS $$
DECLARE s beauty.salon_service; id bigint;
BEGIN
  PERFORM beauty.lock_writes();
  PERFORM beauty.require_actor(p_client, ARRAY['CLIENT']);
  SELECT * INTO s FROM beauty.salon_service
    WHERE salon_service_id = p_service;
  IF NOT FOUND THEN RAISE EXCEPTION 'Service does not exist'
    USING ERRCODE = '23503'; END IF;
  IF s.price IS DISTINCT FROM p_expected_price OR
     s.duration_minutes IS DISTINCT FROM p_expected_duration THEN
    RAISE EXCEPTION 'Service terms changed; confirm again'
      USING ERRCODE = '22023'; END IF;
  INSERT INTO beauty.appointment (
    client_user_id, master_id, salon_service_id, start_at,
    service_name_snapshot, price_snapshot, duration_snapshot
  ) VALUES (
    p_client, p_master, p_service, p_start,
    s.name, s.price, s.duration_minutes
  ) RETURNING appointment_id INTO id;
  RETURN id;
END $$;
```

mark_visit фиксирует COMPLETED или NO_SHOW после окончания посещения. cancel_appointment идемпотентна и сохраняет автора отмены. add_review допускает отзыв клиента о выполненной записи. При активации приглашения проверяются срок и однократность токена.

6. Индексы по прецедентам

В схеме 43 индекса: 23 обеспечивают PK/UNIQUE/EXCLUDE, 19 созданы для запросов и один частичный уникальный индекс защищает открытые приглашения. Индексы ограничений не дублируются.

Индекс	Прецедент	Польза
idx_salon_public_district	П2	Район и идентификатор в опубликованном каталоге.
idx_salon_public_coordinates	П2	Отбор прямоугольника карты; радиус вычисляется для кандидатов.
idx_service_salon_type_price	П2, П3	Тип и цена услуги внутри выбранного салона.
idx_service_type_price	П2	Поиск салонов по типу и диапазону цены одной услуги.
idx_master_salon	П3, П8	Активные мастера салона.
idx_assignment_service	П3, П4	Мастера услуги; обратный порядок к PK связи M:N.
idx_work_master_date	П4, П9, П12	График мастера в нужную дату; INCLUDE окончания.
idx_appointment_client_time	П5	Последние записи клиента без полной сортировки.
idx_appointment_master_time	П9, П10	Журнал мастера за дату и история посещений.
idx_appointment_service_future	П8	Проверка защиты услуги с будущими подтверждёнными записями.
idx_appointment_busy	П4, П12	GiST-поиск пересекающихся подтверждённых интервалов.
idx_review_published_target	П3, П6, П11	Опубликованные оценки нужного типа цели.
idx_salon_photo_visible idx_master_photo_visible	П3	Видимые фото родителя в порядке показа.
idx_moderation_user idx_moderation_salon idx_moderation_master idx_moderation_review	П7, П11	История решений по конкретному объекту; частичные индексы без NULL.
idx_invitation_master_history	П1, П8	История и повторная выдача приглашений мастера.
uq_invitation_open	П1	Не более одного неиспользованного, неотозванного токена.

Примеры индексов для истории клиента и поиска занятых слотов:

```
CREATE INDEX idx_appointment_client_time
  ON beauty.appointment (client_user_id, start_at DESC)
  INCLUDE (status, master_id, salon_service_id);

CREATE INDEX idx_appointment_busy
  ON beauty.appointment USING gist (master_id, slot)
  WHERE status = 'CONFIRMED';
```

7. Заключение

Построены ER-модель из десяти сущностей с M:N и даталогическая модель из четырнадцати отношений. Реализованы PostgreSQL DDL, триггеры, бизнес-операции PL/pgSQL, индексы и скрипты жизненного цикла. Материалы готовы к обсуждению с преподавателем; фактическое согласование ER-модели остаётся обязательным внешним шагом.